DSA Practical – 15 Programs

Simple codes and sample outputs | C, C++ and Python | Based on the uploaded AIML practical question paper

Note: The uploaded PDF contains repeated entries. Practicals 14 and 15 are kept separately as requested. Some language choices are study-friendly suggestions because the question paper does not specify a language for every program.

Practical 1: Student Database using Dynamic Memory Allocation

Language: C

```
#include <stdio.h>
#include <stdlib.h>

struct Student {
    int id;
    char name[50];
    float cgpa;
};

int main() {
    struct Student *s;
    int n, i, key, found = 0;

    printf("Enter number of students: ");
    scanf("%d", &n);

    s = (struct Student*)malloc(n * sizeof(struct Student));

    for(i = 0; i < n; i++) {
        printf("Enter ID, Name and CGPA: ");
        scanf("%d %49s %f", &s[i].id, s[i].name, &s[i].cgpa);
    }

    s = (struct Student*)realloc(s, (n + 1) * sizeof(struct Student));

    printf("Enter new student ID, Name and CGPA: ");
    scanf("%d %49s %f", &s[n].id, s[n].name, &s[n].cgpa);

    printf("\\nStudent Records:\\n");
    for(i = 0; i <= n; i++)
        printf("%d %s %.2f\\n", s[i].id, s[i].name, s[i].cgpa);

    printf("Enter ID for Linear Search: ");
    scanf("%d", &key);
    found = 0;
    for(i = 0; i <= n; i++) {
        if(s[i].id == key) {
            printf("Student Found: %s\\n", s[i].name);
            found = 1;
            break;
        }
    }
    if(!found) printf("Student Not Found\\n");

    // Sort by ID before Binary Search
    struct Student temp;
    for(i = 0; i < n; i++) {
        for(int j = i + 1; j <= n; j++) {
            if(s[i].id > s[j].id) {
                temp = s[i]; s[i] = s[j]; s[j] = temp;
            }
        }
    }

    printf("Enter ID for Binary Search: ");
    scanf("%d", &key);
    int low = 0, high = n, mid;
    found = 0;
    while(low <= high) {
        mid = (low + high) / 2;
        if(s[mid].id == key) {
            printf("Student Found: %s\\n", s[mid].name);
            found = 1; break;
        } else if(s[mid].id < key) low = mid + 1;
        else high = mid - 1;
    }
    if(!found) printf("Student Not Found\\n");

    free(s);
    return 0;
}
```

Sample Output:

```
Input:
2
1 Rahul 8.5
2 Priya 9.0
3 Amit 8.8
2
3

Output:
Student Records:
1 Rahul 8.50
```

```
2 Priya 9.00
3 Amit 8.80
Student Found: Priya
Student Found: Amit
```

Practical 2: Stack using Linked List – Infix to Postfix

Language: C++

```
#include <iostream>
#include <stack>
#include <cctype>
using namespace std;

int priority(char c) {
    if(c == '^') return 3;
    if(c == '*' || c == '/') return 2;
    if(c == '+' || c == '-') return 1;
    return 0;
}

int main() {
    stack<char> s;
    string infix, postfix = "";

    cout << "Enter expression: ";
    cin >> infix;

    for(char c : infix) {
        if(isalnum(c)) postfix += c;
        else if(c == '(') s.push(c);
        else if(c == ')') {
            while(!s.empty() && s.top() != '(') {
                postfix += s.top(); s.pop();
            }
            if(!s.empty()) s.pop();
        } else {
            while(!s.empty() && s.top() != '(' &&
                  priority(s.top()) >= priority(c)) {
                postfix += s.top(); s.pop();
            }
            s.push(c);
        }
    }
    while(!s.empty()) {
        postfix += s.top(); s.pop();
    }

    cout << "Postfix: " << postfix;
    return 0;
}
```

Sample Output:

```
Input:
A+B*C

Output:
Postfix: ABC*+

Note: This is a compact conversion example using the C++ stack library. The paper asks for a stack ADT using a singly li
```

Practical 3: Circular Queue using Array

Language: C++

```
#include <iostream>
using namespace std;

int q[5], front = -1, rear = -1;

void enqueue(int x) {
    if((rear + 1) % 5 == front) {
        cout << "Queue Full\\n";
        return;
    }
    if(front == -1) front = 0;
    rear = (rear + 1) % 5;
    q[rear] = x;
}

void dequeue() {
    if(front == -1) {
        cout << "Queue Empty\\n";
        return;
    }
    cout << "Deleted: " << q[front] << endl;
    if(front == rear) front = rear = -1;
    else front = (front + 1) % 5;
}

void display() {
    if(front == -1) { cout << "Queue Empty"; return; }
    int i = front;
    while(true) {
        cout << q[i] << " ";
        if(i == rear) break;
        i = (i + 1) % 5;
    }
}

int main() {
    enqueue(10); enqueue(20); enqueue(30);
    cout << "Queue: "; display();
    cout << endl;
    dequeue();
    cout << "Queue: "; display();
    return 0;
}
```

Sample Output:

```
Output:
Queue: 10 20 30
Deleted: 10
Queue: 20 30
```

Practical 4: Binary Search Tree – Insert, Delete and Search

Language: C++

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *left, *right;
};

Node* insert(Node* r, int x) {
    if(r == NULL) return new Node{x, NULL, NULL};
    if(x < r->data) r->left = insert(r->left, x);
    else if(x > r->data) r->right = insert(r->right, x);
    return r; // duplicate values are ignored
}

Node* minimum(Node* r) {
    while(r->left != NULL) r = r->left;
    return r;
}

Node* deleteNode(Node* r, int x) {
    if(r == NULL) return r;
    if(x < r->data) r->left = deleteNode(r->left, x);
    else if(x > r->data) r->right = deleteNode(r->right, x);
    else {
        if(r->left == NULL) return r->right;
        if(r->right == NULL) return r->left;
        Node* t = minimum(r->right);
        r->data = t->data;
        r->right = deleteNode(r->right, t->data);
    }
    return r;
}

bool search(Node* r, int x) {
    if(r == NULL) return false;
    if(r->data == x) return true;
    if(x < r->data) return search(r->left, x);
    return search(r->right, x);
}

int main() {
    Node* root = NULL;
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    cout << (search(root, 30) ? "Found" : "Not Found") << endl;
    root = deleteNode(root, 30);
    cout << (search(root, 30) ? "Found" : "Not Found") << endl;
    return 0;
}
```

Sample Output:

```
Output:
Found
Not Found
```

Practical 5: BST – Traversal, Depth and Mirror

Language: C++

```
#include <iostream>
using namespace std;

struct Node { int data; Node *left, *right; };

Node* insert(Node* r, int x) {
    if(r == NULL) return new Node{x, NULL, NULL};
    if(x < r->data) r->left = insert(r->left, x);
    else if(x > r->data) r->right = insert(r->right, x);
    return r;
}

void inorder(Node* r) {
    if(r) { inorder(r->left); cout << r->data << " "; inorder(r->right); }
}

int depth(Node* r) {
    if(r == NULL) return 0;
    int l = depth(r->left), h = depth(r->right);
    return 1 + (l > h ? l : h);
}

void mirror(Node* r) {
    if(r == NULL) return;
    Node* t = r->left; r->left = r->right; r->right = t;
    mirror(r->left); mirror(r->right);
}

int main() {
    Node* root = NULL;
    root = insert(root, 50); root = insert(root, 30); root = insert(root, 70);
    cout << "Traversal: "; inorder(root);
    cout << "\\nDepth: " << depth(root);
    mirror(root);
    cout << "\\nMirror: "; inorder(root);
    return 0;
}
```

Sample Output:

```
Output:
Traversal: 30 50 70
Depth: 2
Mirror: 70 50 30
```

Practical 6: BST – Copy, Parent Nodes, Leaf Nodes and Level-wise Display

Language: C++

```
#include <iostream>
#include <queue>
using namespace std;

struct Node { int data; Node *left, *right; };

Node* insert(Node* r, int x) {
    if(r == NULL) return new Node{x, NULL, NULL};
    if(x < r->data) r->left = insert(r->left, x);
    else if(x > r->data) r->right = insert(r->right, x);
    return r;
}

Node* copyTree(Node* r) {
    if(r == NULL) return NULL;
    Node* n = new Node{r->data, NULL, NULL};
    n->left = copyTree(r->left);
    n->right = copyTree(r->right);
    return n;
}

void parents(Node* r) {
    if(r == NULL) return;
    if(r->left || r->right) {
        cout << "Parent " << r->data << ": ";
        if(r->left) cout << r->left->data << " ";
        if(r->right) cout << r->right->data;
        cout << endl;
    }
    parents(r->left); parents(r->right);
}

void leaves(Node* r) {
    if(r == NULL) return;
    if(r->left == NULL && r->right == NULL) cout << r->data << " ";
    leaves(r->left); leaves(r->right);
}

void levelOrder(Node* r) {
    if(r == NULL) return;
    queue<Node*> q; q.push(r);
    while(!q.empty()) {
        Node* t = q.front(); q.pop();
        cout << t->data << " ";
        if(t->left) q.push(t->left);
        if(t->right) q.push(t->right);
    }
}

int main() {
    Node* root = NULL;
    root = insert(root, 50); root = insert(root, 30); root = insert(root, 70);
    Node* copied = copyTree(root);
    parents(root);
    cout << "Leaf: "; leaves(root);
    cout << "\\nLevel: "; levelOrder(copied);
    return 0;
}
```

Sample Output:

```
Output:
Parent 50: 30 70
Leaf: 30 70
Level: 50 30 70
```

Practical 7: Student Database – Bubble Sort and Insertion Sort

Language: Python

```
students = [
    [1, "Rahul", 8.5],
    [2, "Priya", 9.2],
    [3, "Amit", 8.8]
]

# Bubble Sort by CGPA (ascending)
for i in range(len(students)):
    for j in range(len(students)-1):
        if students[j][2] > students[j+1][2]:
            students[j], students[j+1] = students[j+1], students[j]

print("CGPA Sorting:")
for s in students:
    print(s)

# Insertion Sort by Name
for i in range(1, len(students)):
    key = students[i]
    j = i - 1
    while j >= 0 and students[j][1] > key[1]:
        students[j+1] = students[j]
        j -= 1
    students[j+1] = key

print("Name Sorting:")
for s in students:
    print(s)
```

Sample Output:

```
Output:
CGPA Sorting:
[1, 'Rahul', 8.5]
[3, 'Amit', 8.8]
[2, 'Priya', 9.2]
Name Sorting:
[3, 'Amit', 8.8]
[2, 'Priya', 9.2]
[1, 'Rahul', 8.5]
```

Practical 8: Graph – Minimum Spanning Tree using Kruskal and Prim

Language: Python

```
edges = [
    (1, 2, 10), (1, 3, 6), (2, 3, 5),
    (2, 4, 15), (3, 4, 4)
]

# Kruskal Algorithm
parent = [0, 1, 2, 3, 4]

def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

edges.sort(key=lambda x: x[2])
cost = 0
print("Kruskal MST:")
for u, v, w in edges:
    a, b = find(u), find(v)
    if a != b:
        parent[a] = b
        print(u, "-", v, ":", w)
        cost += w
print("Total Cost:", cost)

# Prim Algorithm
graph = {
    1: [(2,10), (3,6)],
    2: [(1,10), (3,5), (4,15)],
    3: [(1,6), (2,5), (4,4)],
    4: [(2,15), (3,4)]
}
visited = {1}
cost = 0
print("Prim MST:")
while len(visited) < 4:
    best = None
    for u in visited:
        for v, w in graph[u]:
            if v not in visited and (best is None or w < best[2]):
                best = (u, v, w)
    u, v, w = best
    print(u, "-", v, ":", w)
    visited.add(v)
    cost += w
print("Total Cost:", cost)
```

Sample Output:

```
Output:
Kruskal MST:
3 - 4 : 4
2 - 3 : 5
1 - 3 : 6
Total Cost: 15
Prim MST:
1 - 3 : 6
3 - 4 : 4
3 - 2 : 5
Total Cost: 15
```

Practical 9: Heap Sort – Ascending Order

Language: Python

```
def heapify(a, n, i):
    largest = i
    left = 2*i + 1
    right = 2*i + 2

    if left < n and a[left] > a[largest]:
        largest = left
    if right < n and a[right] > a[largest]:
        largest = right

    if largest != i:
        a[i], a[largest] = a[largest], a[i]
        heapify(a, n, largest)

def heapSort(a):
    n = len(a)
    for i in range(n//2 - 1, -1, -1):
        heapify(a, n, i)
    for i in range(n-1, 0, -1):
        a[0], a[i] = a[i], a[0]
        heapify(a, i, 0)

a = [40, 10, 30, 50, 20]
print("Original:", a)
heapSort(a)
print("Sorted:", a)
```

Sample Output:

```
Output:
Original: [40, 10, 30, 50, 20]
Sorted: [10, 20, 30, 40, 50]
```

Practical 10: Merge Sort – Online Orders by Delivery Time

Language: Python

```
def mergeSort(a):
    if len(a) > 1:
        mid = len(a)//2
        left = a[:mid]
        right = a[mid:]
        mergeSort(left)
        mergeSort(right)

        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                a[k] = left[i]
                i += 1
            else:
                a[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            a[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            a[k] = right[j]
            j += 1
            k += 1

orders = [40, 20, 60, 10, 30]
print("Before:", orders)
mergeSort(orders)
print("After:", orders)
```

Sample Output:

```
Output:
Before: [40, 20, 60, 10, 30]
After: [10, 20, 30, 40, 60]
```

Practical 11: Fractional Knapsack – Maximum Profit

Language: Python

```
weight = [10, 20, 30]
profit = [60, 100, 120]
capacity = 50

items = []
for i in range(len(weight)):
    ratio = profit[i] / weight[i]
    items.append((ratio, weight[i], profit[i]))

items.sort(reverse=True)
total = 0

for ratio, w, p in items:
    if capacity >= w:
        total += p
        capacity -= w
    else:
        total += ratio * capacity
        break

print("Maximum Profit:", total)
```

Sample Output:

```
Output:
Maximum Profit: 240.0
```

Practical 12: Naive String Matching Algorithm

Language: Python

```
text = "ABABABC"
pattern = "ABC"

n = len(text)
m = len(pattern)
found = False

for i in range(n - m + 1):
    if text[i:i+m] == pattern:
        print("Pattern found at index:", i)
        found = True

if not found:
    print("Pattern not found")
```

Sample Output:

```
Output:
Pattern found at index: 4
```

Practical 13: Maze Navigation using AI Search – BFS

Language: Python

```
from collections import deque

maze = [
    [0, 0, 1],
    [1, 0, 1],
    [0, 0, 0]
]

start = (0, 0)
goal = (2, 2)
q = deque([(start, [start])])
visited = {start}

while q:
    pos, path = q.popleft()
    x, y = pos

    if pos == goal:
        print("Path Found:", path)
        break

    for dx, dy in [(0,1), (1,0), (0,-1), (-1,0)]:
        nx, ny = x + dx, y + dy
        if (0 <= nx < 3 and 0 <= ny < 3
            and maze[nx][ny] == 0
            and (nx, ny) not in visited):
            visited.add((nx, ny))
            q.append(((nx, ny), path + [(nx, ny)]))
else:
    print("No Path Found")
```

Sample Output:

```
Output:
Path Found: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2)]
```

Practical 14: Repeated Practical – Merge Sort for Online Orders

Language: Python

```
def mergeSort(a):
    if len(a) > 1:
        mid = len(a)//2
        left, right = a[:mid], a[mid:]
        mergeSort(left)
        mergeSort(right)
        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                a[k] = left[i]
                i += 1
            else:
                a[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            a[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            a[k] = right[j]
            j += 1
            k += 1

orders = [50, 20, 40, 10, 30]
mergeSort(orders)
print("Sorted Orders:", orders)
```

Sample Output:

```
Output:
Sorted Orders: [10, 20, 30, 40, 50]
```

Practical 15: Repeated Practical – Fractional Knapsack

Language: Python

```
weight = [10, 20, 30]
profit = [60, 100, 120]
capacity = 50

items = []
for i in range(len(weight)):
    items.append((profit[i]/weight[i], weight[i]))

items.sort(reverse=True)
total = 0

for ratio, w in items:
    if capacity >= w:
        total += ratio * w
        capacity -= w
    else:
        total += ratio * capacity
        break

print("Maximum Profit:", total)
```

Sample Output:

```
Output:
Maximum Profit: 240.0
```